

SMART: Self-Measured Adaptive Routing Tokens for Cost-Efficient LLM Delegation in Mathematical Reasoning

Huy Dang¹ Le Viet Hai^{1,2} Tuan Nguyen^{1,3} Phan Dai¹
¹New Turing Institute ²Huawei ³Salesforce

Abstract

We present SMART (Self-Measured Adaptive Routing Tokens), a novel approach to multi-model delegation that embeds confidence estimation directly within a Small Language Model (SLM) rather than relying on external routing mechanisms. Unlike prior work on LLM cascades that employ separate classifiers or scoring functions for routing decisions, SMART trains special confidence tokens into the SLM’s vocabulary, enabling real-time self-assessment during autoregressive generation. On the MATH-500 benchmark, our framework achieves 83.8% accuracy at \$1.24 total inference cost—a 49.8% cost reduction compared to orchestrator-based delegation while maintaining competitive accuracy. The SLM confidently handles 69% of queries with 86.4% accuracy, substantially exceeding its standalone performance of 67.8%. Our results demonstrate that internal confidence routing offers a more efficient and interpretable alternative to external routing mechanisms, with implications for deploying cost-effective reasoning systems at scale.

Keywords: language model routing, confidence calibration, model cascades, mathematical reasoning, cost-efficient inference

1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities in mathematical reasoning, achieving near-expert performance on challenging benchmarks. However, this capability comes at substantial computational cost—both in terms of inference latency and API pricing. For real-world deployment scenarios requiring millions of queries, the cost differential between a frontier model and a smaller specialized model can be orders of magnitude, creating a fundamental tension between accuracy and economic viability.

This observation has motivated research into model routing and cascade systems, where queries are dynamically assigned to models based on predicted difficulty or confidence. The central insight is that not all queries require frontier-model capabilities—many can be solved accurately by smaller, more efficient models. The challenge lies in accurately predicting which queries require escalation.

Existing approaches to query routing fall into two categories. The first trains external classifiers to predict task difficulty based on input features and historical performance data, forming routing pipelines with multi-agent systems or dedicated router models [Ding et al., 2024, Ong et al., 2024, Stripelis et al., 2024, Jiang et al., 2024, Zhang et al., 2023]. The second employs LLM cascades that invoke increasingly complex models based on scoring functions predicting answer correctness or response consistency upon resampling [Chen et al., 2023, Yue et al., 2023]. Both approaches share a critical limitation: they rely on *external* mechanisms for routing decisions, introducing additional inference overhead and architectural complexity.

We propose SMART (Self-Measured Adaptive Routing Tokens), which fundamentally differs from prior work by training confidence estimation directly into the SLM itself rather than

employing external routing components. Our approach introduces domain-specific confidence tokens (`<|c_math|>` for confident, `<|u_math|>` for uncertain) that the SLM learns to generate after producing an answer. This *internal* routing mechanism offers several advantages over external approaches:

1. The confidence estimate is computed during the autoregressive decoding step, allowing the model to condition on its generated output when assessing certainty.
2. No additional model inference is required for routing decisions, eliminating orchestration overhead entirely.
3. Confidence tokens are aligned against actual correctness rather than proxy measures like response consistency.
4. The routing decision is transparent and interpretable—developers can directly observe why a query was escalated.

Unlike prior work in LLM cascades, SMART trains the estimate of the probability of correctness into the LLM itself instead of an external model. The estimates of confidence from SMART are aligned against the correctness of the predictions, rather than consistency. Additionally, SMART computes the confidence in the autoregressive decoding step, allowing for the LLM itself to condition on its generated output. This dynamic approach allows for more accurate and adaptive routing decisions based on real-time confidence assessments.

Our experiments on MATH-500 demonstrate that SMART achieves 83.8% accuracy at approximately half the cost of orchestrator-based delegation (\$1.24 vs \$2.48), while the SLM’s effective accuracy on self-handled queries improves from 67.8% to 86.4% through confidence-aware filtering. These results establish internal confidence routing as a practical and efficient alternative to external routing mechanisms.

2 Related Work

2.1 Multi-Model Systems and Cascades

The deployment of multiple models in cascade or ensemble configurations has been studied extensively. FrugalGPT [Chen et al., 2023] proposed combining multiple LLM APIs with learned routing to reduce costs while maintaining quality. Speculative decoding [Leviathan et al., 2023] uses small models to draft tokens that larger models verify, accelerating inference without changing outputs. Mixture-of-Experts architectures route different inputs to specialized sub-networks, though routing occurs at the token or layer level rather than the query level.

2.2 LLM Routing and Confidence Estimation

In confidence-based query-routing scenarios, one line of research trains additional classifiers to route queries to LLMs based on observed performance metrics and routing data, forming routing pipelines with trained multi-agent systems or extra routers to boost performance [Ding et al., 2024, Ong et al., 2024, Stripelis et al., 2024, Jiang et al., 2024, Zhang et al., 2023]. Another approach leverages LLM cascades, invoking increasingly complex LLMs depending on trained scoring functions which predict whether an answer is correct, and consistency of responses upon resampling [Chen et al., 2023, Yue et al., 2023].

Unlike prior work in LLM cascades, SMART trains the estimate of the probability of correctness into the LLM itself instead of an external model. The estimates of confidence from SMART are aligned against the correctness of the predictions, rather than consistency. Additionally, SMART computes the confidence in the autoregressive decoding step, allowing for the LLM

itself to condition on its generated output. This dynamic approach allows for more accurate and adaptive routing decisions based on real-time confidence assessments.

2.3 Uncertainty Quantification in LLMs

Recent studies in LLM uncertainty have explored leveraging internal features like logits and attention scores to estimate uncertainty without additional training [Zhou et al., 2023, Mahaut et al., 2024, Turpin et al., 2024]. Post-processing methods such as temperature scaling [Guo et al., 2017], prompting LLMs to verbalize confidence scores [Tian et al., 2023], iterative prompting [Abbasi Yadkori et al., 2024], and instruction learning/pretraining [Zhang et al., 2021, Neeman et al., 2023, Li et al., 2023, Liu et al., 2024, Cohen et al., 2024] have also shown improvements in calibration.

In contrast to these methods, SMART allows the LLM to learn to estimate its confidence at fine-tuning time. Furthermore, this internal notion of confidence is aligned against correctness rather than the consistency of responses after re-sampling, as studied in [Yona et al., 2024, Manakul et al., 2023, Lin et al., 2023]. As noted in [Huang et al., 2023, Wang & Zhou, 2024, Chuang et al., 2025], uncertainty scores derived from LLMs may exhibit a weak correlation with actual prediction correctness, and the most consistent answers are often not the most correct. Finally, while much of the prior work focuses on calibration, our work goes beyond it and focuses on the downstream utility of confidence scores for LLM routing and rejection learning.

2.4 Confidence Calibration in Language Models

Language model calibration—ensuring that expressed confidence correlates with actual accuracy—has received growing attention. Prior work has explored verbalized confidence, where models are prompted to express uncertainty in natural language, and token-level probability analysis. However, these approaches typically treat calibration as an evaluation metric rather than a mechanism for decision-making. Our work operationalizes confidence calibration by training discrete confidence tokens that directly trigger routing decisions.

3 Methodology

3.1 Problem Formulation

We consider a two-model delegation framework consisting of a Small Language Model (SLM) with parameters θ_S and a Large Language Model (LLM) with parameters θ_L . Given a query q from a mathematical reasoning task, the goal is to produce a correct answer y^* while minimizing total inference cost. Let c_S and c_L denote the per-query costs for SLM and LLM inference respectively, where typically $c_L \gg c_S$.

The delegation decision can be formalized as learning a routing function $r : \mathcal{Q} \rightarrow \{\text{SLM}, \text{LLM}\}$ that maximizes expected accuracy subject to a cost budget, or equivalently, minimizes cost subject to an accuracy threshold. The key design question is where this routing function should be implemented.

3.2 Baseline: External Orchestrator (Delegation I)

Our first baseline implements an external orchestrator agent that predicts task difficulty and routes queries accordingly. Given query q , the orchestrator $\mathcal{O}$ produces a routing decision:

$$r(q) = \mathcal{O}(q) \in \{\text{SLM}, \text{LLM}\} \quad (1)$$

This approach suffers from several limitations. First, the orchestrator itself requires inference, adding cost overhead (approximately \$0.126 in our experiments). Second, the orchestrator

must predict difficulty without observing the SLM’s actual attempt at the problem, leading to suboptimal routing—our experiments show roughly 50% of queries are routed to the LLM regardless of actual necessity. Third, the routing decision is opaque, as it emerges from the orchestrator’s internal reasoning.

3.3 SMART: Internal Confidence Routing (Delegation II)

SMART addresses these limitations by embedding the routing decision within the SLM itself. We extend the SLM’s vocabulary with two special tokens: $\langle |c_math| \rangle$ (confident) and $\langle |u_math| \rangle$ (uncertain). After generating an answer $\hat{y}$, the SLM produces a confidence token $t \in \{\langle |c_math| \rangle, \langle |u_math| \rangle\}$.

The routing function becomes:

$$r(q) = \begin{cases} \text{SLM} & \text{if SLM}(q) \text{ ends with } \langle |c_math| \rangle \\ \text{LLM} & \text{if SLM}(q) \text{ ends with } \langle |u_math| \rangle \end{cases} \quad (2)$$

This design has a crucial property: the confidence assessment occurs *after* the SLM has generated its reasoning chain and answer. The model can therefore condition its confidence on the quality and consistency of its own output, rather than predicting difficulty from the query alone.

3.4 Training Pipeline

3.4.1 Confidence-Annotated Dataset Construction

We construct a confidence-annotated training dataset from the MATH benchmark [Hendrycks et al., 2021]. For each problem q with ground-truth answer y^* , we:

1. Generate a step-by-step solution using the base SLM (Qwen 2.5 Math-Instruct 1.5B), producing reasoning chain r and answer $\hat{y}$.
2. Compare $\hat{y}$ against y^* to determine correctness.
3. Append the appropriate confidence token: $\langle |c_math| \rangle$ if correct, $\langle |u_math| \rangle$ if incorrect.

The resulting dataset contains 6,750 training samples, each following the format: $\{\text{system_prompt}\} \{\text{question}\} \{\text{reasoning} + \text{answer}\} \{\text{confidence_token}\}$. The dataset is published on Hugging Face for reproducibility.

3.4.2 LoRA Fine-Tuning

We fine-tune the SLM using Low-Rank Adaptation (LoRA) [Hu et al., 2022] to minimize parameter overhead while enabling effective confidence token prediction. Our configuration:

- Rank $r = 8$
- LoRA alpha = 32
- Dropout = 0.05
- Target modules: `q_proj`, `v_proj`
- Training epochs: 5
- Learning rate: 5×10^{-5}

After fine-tuning, the SLM reliably produces exactly one confidence token at the end of each response, enabling deterministic routing decisions.

Algorithm 1 SMART Inference Protocol

Require: Query q , Fine-tuned SLM, LLM

Ensure: Final answer y

```
1:  $(\hat{y}, t) \leftarrow \text{SLM}(q)$  ▷ Generate answer and confidence token
2: if  $t = <|\text{c\_math}|>$  then
3:   return  $\hat{y}$  ▷ Accept SLM answer
4: else
5:   return  $\text{LLM}(q)$  ▷ Escalate to LLM
6: end if
```

3.5 Inference Protocol

At inference time, the delegation mechanism operates as shown in Algorithm 1.

Note that only uncertain cases incur LLM cost. The SLM inference is always required, but its cost is negligible compared to the LLM (\$0.03 vs \$2.79 for the full dataset in our experiments).

4 Experimental Setup

4.1 Models

We evaluate the following models:

- **SLM:** Qwen 2.5 Math-Instruct 1.5B — A compact model specialized for mathematical reasoning.
- **SLM (7B):** Qwen 2.5 Math-Instruct 7B — A larger SLM variant for comparison.
- **LLM:** Gemini 2.5 Flash — Google’s efficient frontier model.
- **LLM (Lite):** Gemini 2.5 Flash Lite — A cost-reduced variant.
- **LLM (Pro):** Gemini Pro — The highest-capability variant.

4.2 Benchmark

We evaluate on MATH-500, a curated subset of 500 problems from the MATH benchmark [Hendrycks et al., 2021] spanning algebra, geometry, number theory, counting and probability, intermediate algebra, precalculus, and prealgebra. Problems range from competition mathematics at the AMC level to challenging Olympiad-style questions.

4.3 Evaluation Metrics

- **Accuracy:** Percentage of problems where the final system output matches ground truth.
- **Cost:** Total inference cost in USD, computed using Together.AI pricing for SLM and Google API pricing for Gemini models.
- **Confidence Calibration Accuracy:** Percentage of instances where confidence tokens correctly reflect answer correctness.

4.4 Compared Systems

1. **Standalone Models:** SLM-only and LLM-only baselines.
2. **Delegation I (Orchestrator):** External LLM-based routing.
3. **Delegation II (SMART):** Internal confidence-based routing.

Table 1: Performance Comparison on MATH-500. Del. I = Delegation I (Orchestrator), Del. II = SMART.

Metric	1.5B	7B	Flash Lite	Flash	Pro	Del. I	Del. II
Accuracy (%)	67.8	76.4	85.6	88.6	88.8	85.2	83.8
Cost (USD)	\$0.03	\$0.10	\$0.28	\$2.79	\$13.55	\$2.48	\$1.24

Table 2: Delegation I (Orchestrator Agent) Breakdown

Handler	Correct	Total	Accuracy	Cost
LLM (Gemini Flash)	217	248	87.5%	\$2.465
SLM (Qwen 1.5B)	209	252	82.9%	\$0.014

5 Results

5.1 Overall Performance

Table 1 presents comprehensive results across all model configurations and delegation strategies.

Key findings from the overall comparison:

- The standalone 1.5B SLM achieves 67.8% accuracy at minimal cost (\$0.03), while Gemini Pro reaches 88.8% at \$13.55—a $450\times$ cost multiplier for 21 percentage points of accuracy.
- Delegation I (orchestrator-based) achieves 85.2% accuracy at \$2.48, demonstrating that routing can approach frontier-model performance at reduced cost.
- SMART (Delegation II) achieves 83.8% accuracy at \$1.24—a **49.8% cost reduction** compared to Delegation I with only 1.4 percentage points accuracy degradation.

5.2 Delegation Strategy Comparison

Tables 2 and 3 break down performance by which model handled each query under the two delegation strategies.

The orchestrator routes approximately 50% of queries to each model, with weighted accuracy of 85.2%. However, the cost is dominated by LLM inference plus orchestration overhead ($\sim$ \$0.126).

SMART routes only 31% of queries to the LLM, compared to 50% under Delegation I. Critically, the SLM achieves **86.4% accuracy** on the 69% of queries it handles confidently—substantially higher than its standalone accuracy of 67.8%. This demonstrates that confidence-aware filtering effectively identifies queries within the SLM’s competence.

5.3 Cost-Accuracy Analysis

Table 4 directly compares the cost-accuracy trade-offs across all methods.

SMART achieves 94.6% of the LLM’s accuracy at 44.4% of its cost, and 98.4% of Delegation I’s accuracy at 50% of its cost. This represents the Pareto-optimal trade-off point for this benchmark.

5.4 Quantitative Improvements of SMART

We summarize the key quantitative improvements of SMART over Delegation I:

1. **Cost Reduction:** \$1.24 vs \$2.48 — 49.8% cheaper.

Table 3: Delegation II (SMART) Breakdown

Handler	Correct	Total	Accuracy	Cost
LLM (Gemini Flash)	121	155	78.1%	\$1.23
SLM (Qwen 1.5B)	298	345	86.4%	\$0.014

Table 4: Cost-Accuracy Trade-off Analysis

Method	Accuracy	Cost	Notes
LLM-only (Flash)	88.6%	\$2.79	Most accurate
SLM-only (1.5B)	67.8%	\$0.03	Cheapest
Delegation I	85.2%	\$2.48	Near-LLM accuracy
SMART (Ours)	83.8%	\$1.24	Best cost-accuracy ratio

2. **Reduced LLM Dependency:** 31% vs 50% of queries to LLM — 38% fewer LLM calls.
3. **Better SLM Utilization:** 86.4% vs 67.8% accuracy on SLM-handled queries — 18.6 percentage point improvement through confidence-aware filtering.
4. **Planning Overhead Eliminated:** No orchestrator cost ($\sim$ \$0.126 saved).

6 Analysis

6.1 Why Internal Routing Reduces Cost

The cost advantage of SMART stems from multiple factors:

1. **Post-generation confidence assessment:** Because the SLM evaluates confidence after generating its answer, it can make routing decisions based on the quality of its actual output rather than predicted difficulty. This leads to better-calibrated routing.
2. **Self-knowledge through training:** Fine-tuning on correctness-annotated data teaches the SLM its own capabilities. The model learns which types of problems it can solve reliably, enabling accurate self-assessment.
3. **No orchestration overhead:** Delegation I requires an additional LLM call purely for routing, which adds $\sim$ \$0.126 per dataset. SMART eliminates this entirely.
4. **Shorter token sequences:** The SLM produces deterministic confidence tokens rather than verbose routing explanations, reducing total token consumption.

6.2 Interpretability Advantages

Beyond cost efficiency, SMART provides interpretability benefits that external routing cannot match:

- **Transparent routing decisions:** The presence of `<|c_math|>` or `<|u_math|>` explicitly indicates why a query was handled locally or escalated.
- **Debuggable behavior:** Confidence calibration accuracy can be measured directly, enabling systematic improvement.

- **Controllable thresholds:** Developers can post-process confidence tokens or adjust token probabilities to tune the routing threshold without retraining.

In contrast, Delegation I’s orchestrator produces routing decisions through opaque chain-of-thought reasoning, making it difficult to understand or modify routing behavior.

6.3 Limitations and Failure Modes

We observe several limitations of the current approach:

- **Lower LLM accuracy on escalated queries:** The LLM achieves 78.1% accuracy on queries escalated by SMART, compared to 87.5% on queries routed by the orchestrator. This suggests SMART escalates genuinely harder problems, but also that some escalated queries may be beyond even the LLM’s reliable capability.
- **Domain specificity:** The current training uses math-specific confidence tokens. Extending to other domains requires domain-specific annotation and fine-tuning.
- **Binary confidence:** The current approach uses binary confident/uncertain tokens. A continuous confidence score might enable more nuanced routing decisions.

7 Discussion

7.1 Implications for Production Systems

SMART’s cost reduction has significant implications for deploying mathematical reasoning systems at scale. At current pricing, a system processing 1 million queries would cost approximately \$2,480 with SMART versus \$5,580 with LLM-only inference—a savings of over \$3,000 per million queries while maintaining 94.6% of frontier-model accuracy.

The architectural simplicity of SMART also reduces deployment complexity. There is no need to maintain separate routing infrastructure or manage multi-model orchestration—the fine-tuned SLM handles both computation and routing in a single forward pass.

7.2 Future Directions

Several directions could extend this work:

- **Multi-domain confidence tokens:** Training separate confidence tokens for different domains (e.g., `<|c_code|>`, `<|u_physics|>`) could enable a single SLM to route across diverse task types.
- **Continuous confidence scoring:** Replacing binary tokens with scalar confidence predictions could enable probabilistic routing strategies.
- **Multi-tier cascades:** Extending to three or more models with graduated confidence thresholds.
- **Calibration refinement:** Incorporating techniques like temperature scaling or Platt scaling to improve confidence calibration post-training.

8 Conclusion

We presented SMART (Self-Measured Adaptive Routing Tokens), a framework for multi-model delegation that embeds confidence estimation directly within the Small Language Model rather than relying on external routing mechanisms. By training domain-specific confidence tokens that the SLM generates after producing an answer, SMART enables real-time self-assessment during autoregressive decoding.

On the MATH-500 benchmark, SMART achieves 83.8% accuracy at \$1.24 total cost—a 49.8% cost reduction compared to orchestrator-based delegation while maintaining competitive accuracy. The SLM confidently handles 69% of queries with 86.4% accuracy, demonstrating that internal confidence routing effectively identifies queries within the model’s competence.

Our results establish internal confidence routing as a practical and efficient alternative to external routing mechanisms. The approach offers not only cost advantages but also improved interpretability, as routing decisions are explicitly encoded in the model’s output rather than emerging from opaque orchestrator reasoning.

As language model deployment scales to millions of queries, the cost differential demonstrated here becomes economically significant. SMART provides a principled approach to achieving this efficiency while maintaining the transparency needed for reliable production systems.

Acknowledgements

Our sincere gratitude to the GStar Bootcamp Organizers, our lecturers, mentors, and peers—especially Long Phan, for prompting us with the idea! We also thank our dedicated teammates for their contributions throughout this project.

Author Contributions. Throughout this process, our team experimented with many different ideas, some of which were not presented in this paper. However, no work goes unnoticed.

Huy Dang: Confidence token data preparation pipeline; Confidence Routing SFT setup; LLM → SLM delegation on GSM8K (not shown).

Le Viet Hai: LoRA fine-tuning for confidence; agent orchestration baseline; LLM → SLM delegation on MATH-500 (not shown).

Phan Dai: GRPO fine-tuning for confidence token (not shown); LLM → SLM delegation on multi-hop QA (not shown); LLM → SLM delegation fine-tuning on multi-hop QA (not shown).

Tuan Nguyen: LLM → SLM delegation on SWE-Bench (not shown); SFT, DPO, GRPO runs on LLM’s verbalized confidence calibration (not shown).

References

- Chen, L., Zaharia, M., & Zou, J. (2023). FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176*.
- Ding, Y., et al. (2024). Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing. *arXiv preprint arXiv:2404.14618*.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., & Steinhardt, J. (2021). Measuring Mathematical Problem Solving With the MATH Dataset. *NeurIPS 2021*.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022*.
- Jiang, A. Q., et al. (2024). MixEval: Deriving Wisdom of the Crowd from LLM Benchmark Mixtures. *arXiv preprint arXiv:2406.06565*.

- Leviathan, Y., Kalman, M., & Matias, Y. (2023). Fast Inference from Transformers via Speculative Decoding. *ICML 2023*.
- Ong, I., et al. (2024). RouteLLM: Learning to Route LLMs with Preference Data. *arXiv preprint arXiv:2406.18665*.
- Stripelis, D., et al. (2024). TensorOpera Router: A Multi-Model Router for Efficient LLM Inference. *arXiv preprint arXiv:2408.05039*.
- Yue, X., et al. (2023). Automatic Model Selection with Large Language Models for Reasoning. *EMNLP 2023 Findings*.
- Zhang, S., et al. (2023). LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion. *ACL 2023*.
- Zhou, K., et al. (2023). Navigating the Grey Area: Expressions of Overconfidence and Uncertainty in Language Models. *arXiv preprint arXiv:2302.13439*.
- Mahaut, M., Roit, L., Yin, Y., & Tsarfaty, R. (2024). Factual Confidence of LLMs: On Reliability and Robustness of Current Estimators. *arXiv preprint arXiv:2406.10524*.
- Turpin, M., Michael, J., Perez, E., & Bowman, S. R. (2024). Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. *NeurIPS 2023*.
- Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. *ICML 2017*.
- Tian, K., Mitchell, E., Yao, H., Manning, C. D., & Finn, C. (2023). Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback. *EMNLP 2023*.
- Abbasi Yadkori, Y., et al. (2024). Mitigating LLM Hallucinations via Conformal Abstention. *arXiv preprint arXiv:2404.10960*.
- Zhang, S., et al. (2021). Knowing What You Know: Calibrating Dialogue Belief State Distributions via Ensembles. *Findings of EMNLP 2021*.
- Neeman, E., Aharoni, R., Honovich, O., Choshen, L., Szpektor, I., & Abend, O. (2023). DisentQA: Disentangling Parametric and Contextual Knowledge with Counterfactual Question Answering. *ACL 2023*.
- Li, S., et al. (2023). Making Language Models Better Reasoners with Step-Aware Verifier. *ACL 2023*.
- Liu, H., et al. (2024). LitCab: Lightweight Language Model Calibration over Short- and Long-form Responses. *arXiv preprint arXiv:2406.01016*.
- Cohen, R., Biran, M., Yoran, O., Globerson, A., & Berant, J. (2024). LM vs LM: Detecting Factual Errors via Cross-Examination. *EMNLP 2023*.
- Yona, G., Aharoni, R., & Geva, M. (2024). Narrowing the Knowledge Evaluation Gap: Open-Domain Question Answering with Multi-Granularity Answers. *arXiv preprint arXiv:2401.04695*.
- Manakul, P., Liusie, A., & Gales, M. J. F. (2023). SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. *EMNLP 2023*.

- Lin, S., Hilton, J., & Evans, O. (2023). Teaching Models to Express Their Uncertainty in Words. *TMLR 2022*.
- Huang, J., et al. (2023). Look Before You Leap: An Exploratory Study of Uncertainty Measurement for Large Language Models. *arXiv preprint arXiv:2307.10236*.
- Wang, X., & Zhou, D. (2024). Chain-of-Verification Reduces Hallucination in Large Language Models. *arXiv preprint arXiv:2309.11495*.
- Chuang, Y.-S., et al. (2025). Confident Natural Language Inference. *arXiv preprint arXiv:2406.00138*.

A Extended Results with Lite Models

We also evaluated SMART using Gemini 2.5 Flash Lite as the LLM backend. This configuration achieves 81.6% accuracy at \$0.157 total cost, demonstrating that the framework scales effectively across different LLM capability tiers.

Table 5: Lite Configuration Comparison

Configuration	Accuracy	Cost	LLM Backend
Delegation I (Lite)	81.8%	\$0.35	Flash Lite
SMART (Lite)	81.6%	\$0.157	Flash Lite
Delegation I (Full)	85.2%	\$2.48	Flash
SMART (Full)	83.8%	\$1.24	Flash

The Lite configuration offers an even more aggressive cost-accuracy trade-off, achieving 81.6% accuracy at under \$0.16—approximately 5.6% of LLM-only Flash cost while retaining 92% of its accuracy.

B Dataset Details

The confidence-annotated training dataset was constructed from the full MATH training set. After filtering for successful SLM generations (valid reasoning chains and extractable answers), the final dataset contains 6,750 samples with the following distribution:

- Correct answers (annotated with `<|c_math|>`): $\sim 4,590$ samples (68%)
- Incorrect answers (annotated with `<|u_math|>`): $\sim 2,160$ samples (32%)

This distribution reflects the base SLM’s accuracy on the training set and ensures the model sees sufficient examples of both confident and uncertain scenarios during training. The dataset is publicly available at Hugging Face under `Akirayasha/math-20`.